[승코의 차근차근 Flutter] 05. 상태관리

🧠 1. 상태(State)란 무엇일까?

✅ 상태란?

> 시간에 따라 변하는 데이터를 뜻해.
>
> 예를 들면 이런 것들이야:

- 버튼을 눌렀을 때 숫자가 증가하는 `카운터`
- 로그인 여부 (`로그인됨 / 로그아웃됨`)
- 사용자가 작성한 텍스트
- 서버에서 받아온 데이터 목록 등등...

> 📌 Flutter 앱에서는 "상태가 바뀌면 UI가 다시 그려진다." → 이게 핵심 원리야!

🧱 2. StatelessWidget vs StatefulWidget

🟦 2.1 StatelessWidget (상태가 없는 위젯)

> 한 번 생성되면 절대 바뀌지 않는 위젯이야.

```
class MyStateless extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Text('안녕하세요'); // 항상 같은 텍스트만 보여줌
  }
}
```

📌 특징

- 상태를 가지지 않음
- **오직 build()만 실행**됨
- 매개변수로 전달받은 값으로만 동작
- 변경하려면 부모가 다시 새로 만들어야 함

🟨 2.2 StatefulWidget (상태를 가질 수 있는 위젯)

> 내부 상태에 따라 UI가 달라지는 위젯이야.

```
class MyCounter extends StatefulWidget {
  @override
  State<MyCounter> createState() => _MyCounterState();
```

```
}

class _MyCounterState extends State<MyCounter> {
  int counter = 0;

  void _increase() {
    setState(() {
      counter++; // 상태 변경
    });
  }

  @override
  Widget build(BuildContext context) {
    return Column(
      children: [
        Text('$counter'),
        ElevatedButton(onPressed: _increase, child: Text('증가')),
      ],
    );
  }
}
```

📌 특징

- `StatefulWidget`은 UI + 상태(State)를 관리함
- `setState()`를 호출하면 UI가 다시 그려짐
- 클릭, 입력, API 결과 등 실시간 변화가 필요한 경우 사용

🔄 3. setState()란?

> 상태(State)를 바꾼 뒤, 그 변화를 Flutter에게 알려주는 함수야.

```
setState(() {
  counter++; // 데이터 변경
});
```

- 내부적으로 build() 함수를 다시 호출해서 새로운 화면을 그림
- 무조건 모든 위젯을 새로 그리는 게 아니라 **변경된 부분만** 다시 그림 (렌더링 최적화되어 있음)

🧩 4. 예제 비교

📌 StatelessWidget 예시

```
class HelloText extends StatelessWidget {
  final String name;
  HelloText({required this.name});

  @override
  Widget build(BuildContext context) {
    return Text('안녕하세요, $name');
  }
}
```

- 외부에서 전달받은 name이 변하지 않으면, 이 위젯도 바뀌지 않아.
- **UI는 고정적**이고 변화 없음.

📌 StatefulWidget 예시 (카운터 앱)

```
class CounterApp extends StatefulWidget {
  @override
  State<CounterApp> createState() => _CounterAppState();
}
```

```
class _CounterAppState extends State<CounterApp> {
  int count = 0;

  void increase() {
    setState(() {
      count += 1;
    });
  }

  @override
  Widget build(BuildContext context) {
    return Column(
      children: [
        Text('$count'),
        ElevatedButton(
          onPressed: increase,
          child: Text('증가'),
        )
      ],
    );
  }
}
```

- 버튼 클릭마다 `setState()`로 `count`를 바꾸고, Text 위젯이 다시 그려짐

🧠 5. 언제 Stateless, 언제 Stateful 쓸까?

상황	StatelessWidget	StatefulWidget
고정된 UI	✅ 사용	❌ 필요 없음
버튼, 입력 등 UI 변화 필요	❌ 안 됨	✅ 꼭 필요
외부 데이터를 보여주기만 할 때	✅ 사용	❌ 필요 없음
애니메이션, 로딩 상태, 토글	❌ 안 됨	✅ 꼭 필요

📦 6. 상태 관리(State Management)란?

> 단순한 setState만으로 앱의 모든 상태를 관리하기에는 복잡하고 버그가 생길 수 있어.
>
> → 그래서 **상태를 따로 관리하는 방식**을 사용해.

✅ 대표적인 Flutter 상태관리 방식

방식	설명	특징
`setState()`	가장 기본	작고 간단한 앱에 적합
`Provider`	공식 권장	간단하지만 확장성 높음
`Riverpod`	Provider 업그레이드	test-friendly, 강력함
`Bloc`, `Cubit`	이벤트 기반 구조	명확한 분리, 대규모에 적합
`GetX`, `MobX` 등	써드파티 방식	간단한 문법, 다양한 기능

✅ 예시 시나리오: 커피 주문 앱

상태 종류	설명	Stateful 사용?	Provider?
버튼 클릭 시 수량 증가	단일 화면, 간단	✅	❌
메뉴 리스트 불러오기	서버 통신 필요	❌	✅
로그인 상태 유지	앱 전체에 적용	❌	✅
다크모드 설정 저장	어디서나 사용	❌	✅

🛠️ 7. 실전 팁

- 작은 변화, 화면 단위 → `StatefulWidget`
- 데이터가 앱 전체에 퍼짐 → `Provider`, `Riverpod` 등 사용
- `setState()`는 리빌드 범위를 좁게 유지하자
- `StatefulWidget` 내에서 너무 많은 로직 넣지 말고, 분리하자

✅ 정리 요약

구분	설명
State	시간이 지나며 변할 수 있는 데이터
StatelessWidget	상태가 없고 고정된 UI를 만듦
StatefulWidget	상태가 있고, `setState` 로 UI를 갱신함
setState()	상태 변경을 Flutter에게 알리는 함수
상태관리	앱 전역에서 데이터를 효율적으로 관리하는 구조
Provider	Flutter 공식 권장, 구조적 상태관리 도구